

Instituto Tecnológico de Estudios Superiores de
Occidente



ITESO, Universidad Jesuita de Guadalajara

Internet of Things (IoT)
Professors: Mario Castañeda and Ricardo Zamora Dávalos

Lab 2: MQTT

Ana Isabel Guillen González
Diego Fernando Pérez González

October 2025

Table of Contents

Introduction	3
Theoretical Framework	3
TCP	3
Heartbeat Mechanism	3
MQTT Client	4
MQTT Server (Broker)	4
MQTT Packets	4
CONNECT	4
CONNACK	4
PUBLISH	4
SUBSCRIBE	5
Development	5
Code explanation	5
MQTT_Client	5
Server	7
mqtt_frame	7
Steps to send source code to board, connect to network, and start sending Packets	8
Results	9
Conclusions	9
References	9

Introduction

MQTT (Message Queuing Telemetry Transport) was presented to us prior to the development and implementation of this lab, where the final objective was to implement MQTT clients that could publish and subscribe to a network managed by an MQTT server (broker). During the introduction of the protocol and the development of the lab, we learned basic features and key components of the protocol.

MQTT considered a “lightweight” communication protocol because is minimal frame overhead and connection complexity, targeting then IoT systems for example. The main characteristic that stood out to us is that it uses a publish-subscribe and topics model, where client publishers send specific messages to certain topics (humidity percentage, temperature sensor readings, etc. can be really anything) and client subscribers receive messages from specific topics of interest. All this communication handled by a broker server.

Throughout this document a little bit of theoretical background is explained, as well as what we did and tried to do to achieve the final objectives of the lab which were to be able to subscribe and receive messages, as well as to publish and send messages through the broker server. Finally, we present what work and didn't work (results), as well as lessons and conclusions.

Theoretical Framework

TCP

Communication standard, used in MQTT, that enables application programs and computing devices to exchange messages over a network. Designed to send packets across and ensure the successful delivery of data. It organizes data so it can be transmitted between server and a client. It guarantees the integrity of the data being communicated (before it transmits data, TCP established a connection between a source and a destination) this protocol is an alternative to UDP for example, being this last one mainly focused on achieving low latency rather than data integrity [2].

Heartbeat Mechanism

This MQTT mechanism can be seen as a keep alive interval, uses PINGREQ and PINGRESP packets (explained in this theoretical framework) to maintain a connection between a client and a broker. The client sends periodically a PINGREQ packet to signal its presence, the broker responds with a PINGRESP. If the broker does not receive any message (including a PINGREQ) from the client within 1.5 times the keep alive interval, it will assume the connection is lost and disconnects the client [3].

MQTT Client

A program or device that uses MQTT. It always establishes the Network Connection to the Server [4]. It can:

- Publish Application Messages that other Clients might be interested in.
- Subscribe to request Application Messages that it is interested in receiving.
- Unsubscribe to remove a request for Application Messages.
- Disconnect from the Server

MQTT Server (Broker)

A program or device that acts as an intermediary between Clients which publish Application Messages and Clients which have made Subscriptions [4]. A Server:

- Accepts Network Connections from Clients
- Accepts Application Messages published by Clients.
- Processes Subscribe and Unsubscribe requests from Clients.
- Forwards Application Messages that match Client Subscriptions.

MQTT Packets

MQTT control packets represent the basic unit of data transfer. Exchanged between broker servers and MQTT clients to perform various functions, including subscribing to topics and publishing messages [5].

CONNECT

Used to initiate a connection, where the client sends a command message to the broker. Is how the client request to connect to the broker basically. ID/type is 0x01.

CONNACK

When a broker receives a CONNECT message, it is obligated to respond with a CONNACK message. This message contains two data entries: a session present flag and a return code which lets the client know if the connection was accepted or if it wasn't for some of reasons like unacceptable protocol version, identifier rejection, unavailable server, bad username/password or not authorization [4].

PUBLISH

Is a message sent by a client to an MQTT broker, containing a topic name and the actual message payload. This message must contain a topic name [6].

SUBSCRIBE

Used to receive only messages on topics of interest, this is achieved by the client sending a SUBSCRIBE message to the MQTT broker. The SUBSCRIBE message contains a unique packet identifier, 8, and a list of subscriptions [4].

Development

Code explanation

MQTT_Client

The MQTT_Client.c is an implementation that connects to an MQTT broker, can either publish a message to a topic, or subscribe to a topic to receive messages.

Connection:

Using a TCP socket, we connect to a broker at IP address/argv[1] on port 1883. Then we send a CONNECT packet to initiate the MQTT session, sequentially waiting for a CONNACK Response as well:

```
struct sockaddr_in serveraddr;
int sockfd;

sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd < 0)
{
    perror("socket");
    return 1;
}

memset(&serveraddr, 0, sizeof(serveraddr));
serveraddr.sin_family = AF_INET;
serveraddr.sin_port = htons(1883);

if (inet_pton(AF_INET, ipBroker, &serveraddr.sin_addr) <= 0)
{
    perror("IP invalida");
    return 1;
}

if (connect(sockfd, (struct sockaddr *)&serveraddr, sizeof(serveraddr)) < 0)
{
    perror("connect");
    return 1;
}

printf("Conectado a Mosquitto en %s!\n", ipBroker);
```

Illustration 1. Creation and connection of TCP Socket

```
sConnect frame;
frame.bFrameType = 0x10;
frame.wProtNameLen = 4;
memcpy(frame.sProtName, "MQTT", 4);
frame.bVersion = 4;
frame.bConnectFlags = 0x02; // Clean session
frame.bKeepAlive = 60;
frame.wClientIdLen = 5;
memcpy(frame.sClientID, "C1D01", 5);
frame.wMsgLen = 2 + 4 + 1 + 1 + 2 + 2 + frame.wClientIdLen;

sendConnect(sockfd, &frame);
printf("CONNECT enviado\n");
```

Illustration 2. Build and Send MQTT CONNECT Packet

```
sConnAck connack;
int n = recv(sockfd, &connack, sizeof(connack), 0);
if (n != sizeof(connack) || connack.bReturnStatus != 0)
{
    printf("Error de conexion MQTT\n");
    close(sockfd);
    return 1;
}
printf("Conexion MQTT aceptada!\n");
```

Illustration 3. Receive of CONNACK Response

Publish or Subscribe:

Based on the second command-line (better explained at the steps section), argv[2], we either Publish or subscribe. The sections that handles publishing and subscribing are the following:

```
if (strcmp(action, "publish") == 0)
{
    sendPublish(sockfd, topic, msg);
    printf("Mensaje PUBLISH enviado al topic '%s'\n", topic);
}
```

Illustration 4. Publishing a Message

If the user runs the program with "publish" as the second argument, it sends a **PUBLISH** packet to the specified topic with the given message.

sendPublish() constructs the MQTT PUBLISH packet with QoS 0 and sends it over the socket.

```
else if (strcmp(action, "receive") == 0)
{
    uint16_t packetID = 1;
    sendSubscribe(sockfd, topic, packetID);
    printf("SUBSCRIBE enviado para topic '%s'\n", topic);
    printf("Escuchando mensajes del topic '%s'...\n", topic);
    char recvTopic[TOPIC_MAX];
    char recvPayload[PAYLOAD_MAX];

    while (1)
    {
        int len = recvPublishWithPing(sockfd, recvTopic, recvPayload, 60);
        if (len > 0)
        {
            printf("Mensaje recibido: Topic='%s', Payload='%s'\n", recvTopic, recvPayload);
        }
    }
}
```

Illustration 5. Subscribing to a Topic

If the user runs the program with "receive" as the second argument, it sends a SUBSCRIBE packet to the specified topic.

Then it enters a loop, listening for incoming PUBLISH messages.

It also sends PINGREQ messages periodically to keep the connection alive.

Summary of MQTT Packet Construction functions:

- sendConnect() – builds and sends a CONNECT packet.
- sendPublish() – builds and sends a PUBLISH packet (QoS 0).
- sendSubscribe() – builds and sends a SUBSCRIBE packet.
- sendPing() – sends a PINGREQ to keep the connection alive.
- recvPublishWithPing() – listens for incoming PUBLISH messages and sends periodic PINGREQs if no messages are received.

Server

Just for development purposes, in the sense that this script sets up a basic TCP server that listens for incoming connections. It helped us to be sure that our client was working before testing with the broker MQTT server of the teacher at the end of the lab.

mqtt_frame

The mqtt_frame.h is where we defined C structures that represent the different types of MQTT protocol frames, which are used in the MQTT_Client.c code to build and interpret the MQTT messages:

```
typedef struct
{
    uint8_t bFrameType;
    uint16_t wMsgLen;
    uint16_t wProtNameLen;
    char sProtName[4];
    uint8_t bVersion;
    uint8_t bConnectFlags;
    uint16_t bKeepAlive;
    uint16_t wClientIdLen;
    char sClientID[100];
} sConnect;
```

Illustration 6. MQTT CONNECT packet structure frame

```
typedef struct {
    uint8_t bFrameType;
    uint8_t bMsgLen;
    uint16_t wPacketID;
    uint16_t wTopicLen;
    char sTopic[100];
    uint8_t bQoS;
} sSubscribe;
```

Illustration 7. MQTT SUBSCRIBE packet structure frame

```
typedef struct
{
    uint8_t bFrameType;
    uint8_t bMsgLen;
    uint8_t bFlags;
    uint8_t bReturnStatus;
} sConnAck;
```

Illustration 8. MQTT CONNACK response frame (from the broker after a CONNECT)

```
typedef struct
{
    uint8_t bFrameType;
    uint8_t bMsgLen;
} sPingRE;
```

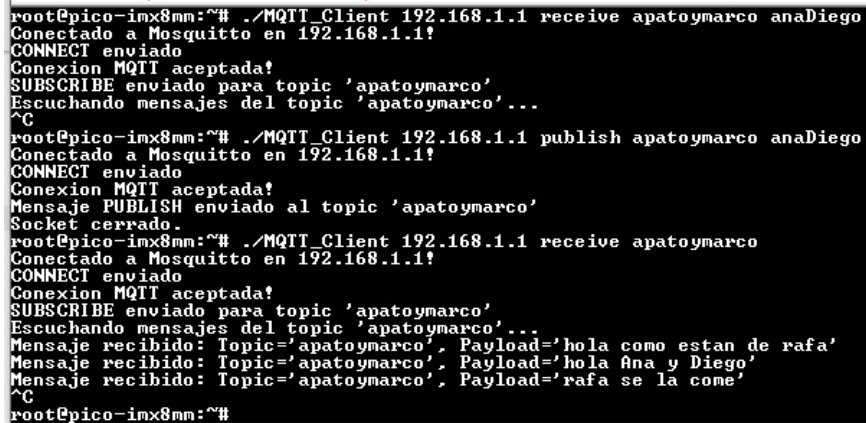
Illustration 9. Frame structure for either a PINGREQ or a PINGRESP

Steps to send source code to board, connect to network, and start sending Packets

1. We send the binary file to our board (a PICO-PI-IMX8M-MINI)
2. We connect the board to the Wi-Fi network:
This was achieved using a Marvell Wi-fi module and obtaining the IP address by executing the following commands:
 - **modprobe moal mod_para=nxp/wifi_mod_para.conf** (loads the Marvell Wi-Fi driver into the linux kernel)
 - **wpa_passphrase MQTT_Broker 123456789 > /etc/wpa_supplicant.conf** (generates a WPA configuration file for connecting to a Wi-Fi network)
 - **wpa_supplicant -d -B -i wlan0 -c /etc/wpa_supplicant.conf -Dnl80211** (starts the daemon to manage the Wi-Fi connection)
 - **udhcpc -i wlan0** (starts the DHCP client to obtain an IP address for the wlan0 interface)
3. Run/execute the program (to either subscribe or publish something):
You just type in on the terminal the: binary file -> broker's IP -> MQTT Packet/action to do -> the network Topic -> message (if applicable)

```
root@pico-imx8mn:~# ./MQTT_Client 192.168.1.1 publish apatoymarco anaDiego
```


Results

A terminal window showing the execution of an MQTT client. The user enters commands to connect, subscribe, and publish messages. The terminal output shows the client's response to each command, including connection status, subscription confirmation, and received messages.

```
root@pico-inx8mm:~# ./MQTT_Client 192.168.1.1 receive apatoymarco anaDiego
Conectado a Mosquitto en 192.168.1.1!
CONNECT enviado
Conexion MQTT aceptada!
SUBSCRIBE enviado para topic 'apatoymarco'
Escuchando mensajes del topic 'apatoymarco'...
^C
root@pico-inx8mm:~# ./MQTT_Client 192.168.1.1 publish apatoymarco anaDiego
Conectado a Mosquitto en 192.168.1.1!
CONNECT enviado
Conexion MQTT aceptada!
Mensaje PUBLISH enviado al topic 'apatoymarco'
Socket cerrado.
root@pico-inx8mm:~# ./MQTT_Client 192.168.1.1 receive apatoymarco
Conectado a Mosquitto en 192.168.1.1!
CONNECT enviado
Conexion MQTT aceptada!
SUBSCRIBE enviado para topic 'apatoymarco'
Escuchando mensajes del topic 'apatoymarco'...
Mensaje recibido: Topic='apatoymarco', Payload='hola como estan de rafa'
Mensaje recibido: Topic='apatoymarco', Payload='hola Ana y Diego'
Mensaje recibido: Topic='apatoymarco', Payload='rafa se la come'
^C
root@pico-inx8mm:~#
```

Illustration 10. Broker Connection Evidence (Connect, Subscribe, Publish Control Packets)

Conclusions

This lab was quite challenging for us, as it basically consisted of learning/hearing about a new communication protocol like MQTT for the first time in the first class, and finishing developing a client code capable of connecting to a network managed by an MQTT server (broker) in the second class, a fairly short time. Working under pressure was quite challenging, so much so that at the end of the class we even had to receive support from a classmate to connect, subscribe, and publish to the network. However, we can satisfactorily say that we finished this lab with a more solid understanding of what MQTT is, its key aspects, how it works, and the basic essence of the protocol consisting of the dynamics of subscription and publication through topics.

References

- [1] *MQTT Version 3.1.1*. (n.d.). https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html#_Toc398718035
- [2] *What is TCP/IP in Networking?* | Fortinet. (n.d.). Fortinet. [https://www.fortinet.com/resources/cyberglossary/tcp-ip#:~:text=Transmission%20Control%20Protocol%20\(TCP\)%20is,data%20and%20messages%20over%20networks.](https://www.fortinet.com/resources/cyberglossary/tcp-ip#:~:text=Transmission%20Control%20Protocol%20(TCP)%20is,data%20and%20messages%20over%20networks.)
- [3] *MQTT Application Note - QuecPython*. (n.d.). https://developer.quectel.com/doc/quecpython/Application_guide/en/network-comm/net-protocols/mqtt.html
- [4] García, M. (2025). *MQTT Protocol Overview*. [Class lecture notes]. ITESO University

[5] Hivemq Team. (2024, July 22). *MQTT Packets: A Comprehensive guide*. Hivemq.
<https://www.hivemq.com/blog/mqtt-packets-comprehensive-guide/>

[6] Team, E. (n.d.). *MQTT Control Packets: A Beginner's Guide*. [www.emqx.com](https://www.emqx.com/en/blog/introduction-to-mqtt-control-packets).
<https://www.emqx.com/en/blog/introduction-to-mqtt-control-packets>